

Problem A. Wooden Spoon

Time Limit 4000 ms

Mem Limit 524288 kB

2^n people, numbered with distinct integers from 1 to 2^n , are playing in a single elimination tournament. The bracket of the tournament is a full binary tree of height n with 2^n leaves.

When two players meet each other in a match, a player with the **smaller** number always wins. The winner of the tournament is the player who wins all n their matches.

A virtual consolation prize "Wooden Spoon" is awarded to a player who satisfies the following n conditions:

- they lost their first match;
- the player who beat them lost their second match;
- the player who beat that player lost their third match;
- ...;
- the player who beat the player from the previous condition lost the final match of the tournament.

It can be shown that there is always exactly one player who satisfies these conditions.

Consider all possible $(2^n)!$ arrangements of players into the tournament bracket. For each player, find the number of these arrangements in which they will be awarded the "Wooden Spoon", and print these numbers modulo 998 244 353.

Input

The only line contains a single integer n ($1 \leq n \leq 20$) — the size of the tournament.

There are 20 tests in the problem: in the first test, $n = 1$; in the second test, $n = 2$; ...; in the 20-th test, $n = 20$.

Output

Print 2^n integers — the number of arrangements in which the "Wooden Spoon" is awarded to players $1, 2, \dots, 2^n$, modulo 998 244 353.

Examples

Input	Output
1	0 2

Input	Output
2	0 0 8 16

Input	Output
3	0 0 0 1536 4224 7680 11520 15360

Note

In the first example, the "Wooden Spoon" is always awarded to player 2.

In the second example, there are 8 arrangements where players 1 and 4 meet each other in the first match, and in these cases, the "Wooden Spoon" is awarded to player 3. In the remaining 16 arrangements, the "Wooden Spoon" is awarded to player 4.

Problem B. Maximum Polygon

Time Limit 3000 ms

Mem Limit 262144 kB

Given an array a of length n , determine the lexicographically largest* subsequence† s of a such that s can be the side lengths of a polygon.

Recall that s can be the side lengths of a polygon if and only if $|s| \geq 3$ and

$$2 \cdot \max(s_1, s_2, \dots, s_{|s|}) < s_1 + s_2 + \dots + s_{|s|}.$$

If no such subsequence s exists, print -1 .

*A sequence x is lexicographically smaller than a sequence y if and only if one of the following holds:

- x is a prefix of y , but $x \neq y$;
- in the first position where x and y differ, the sequence x has a smaller element than the corresponding element in y .

†A sequence x is a subsequence of a sequence y if x can be obtained from y by deleting several (possibly zero or all) elements.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($3 \leq n \leq 2 \cdot 10^5$) — the length of the array a .

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the array a .

It is guaranteed that the total sum of all values of n across all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the answer in the following format:

If the answer exists, output the following.

In the first line, output the integer k ($1 \leq k \leq n$) — the length of the subsequence s .

In the second line, output k integers $s_1, s_2, \dots, s_k$ ($1 \leq s_i \leq 10^9$, s is a subsequence of a) — the subsequence s . Note that the desired output is the value, not the index.

Otherwise, output a single line with the integer -1 .

Examples

Input	Output
5	-1
3	3
3 1 2	4 2 3
4	4
1 4 2 3	6 5 3 2
6	5
1 6 4 5 3 2	43 99 53 22 4
6	4
43 12 99 53 22 4	54 73 23 1
7	
9 764 54 73 22 23 1	

Note

In the first test case, there are no subsequences that can be the side lengths of a polygon.

In the second test case, there are 2 subsequences that can be the side lengths of a polygon: 1, 4, 2, 3 and 4, 2, 3. The latter is the lexicographically larger subsequence.

Problem C

DNA Subsequences



The 3rd Universal Cup, Stage 40: Potyczki. Limits: 1024 MB, 3 s.

21.06.2025

In this task, we consider sequences of nucleotides in a DNA molecule, which are strings composed of characters 'A', 'C', 'G', and 'T'. For each natural number k , there are 4^k different k -letter nucleotide sequences. For a fixed natural number k , we say that a given nucleotide sequence s is k -rich if all k -letter nucleotide sequences are subsequences of s (not necessarily contiguous).

You are given an n -letter nucleotide sequence s . For each natural number k in the range $[1, n]$, output the minimum number of characters that must be changed in s to make it a k -rich sequence. Note that for each k , the result is calculated independently.

Input

The first line of the input contains an integer n ($1 \leq n \leq 200\,000$), representing the length of the string s .

The second line of the input contains an n -letter nucleotide sequence s , consisting only of characters 'A', 'C', 'G', and 'T'.

Output

The output should consist of n integers; the k -th integer should represent the minimum number of characters that must be changed in s to make s a k -rich nucleotide sequence. If it is impossible to change the characters in s in the described way for a given k , then the k -th number should be -1 instead.

Example

For the input data:

8

AAGTAGAA

the correct result is:

1 3 -1 -1 -1 -1 -1 -1

Explanation: For $k = 1$, we can change s with one modification to, for example, AAGTCGAA. The resulting nucleotide sequence then contains all one-letter words as subsequences (in other words, each of the four letters appears at least once), and thus is 1-rich.

For $k = 2$, we can change s with three modifications to, for example, a 2-rich nucleotide sequence CAGTTGAC. Note that we could not change s to, for example, the sequence CCGTTGAA, as it does not contain the two-letter word AC as a subsequence.

For $k > 2$, it is impossible to make the sequence s k -rich.

Best Friend, Worst Enemy

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 32 megabytes

Bobo is analyzing a group of n people, where the i -th ($1 \leq i \leq n$) person has two attributes, x_i and y_i . The attribute values of different people are distinct. For any two persons $1 \leq i, j \leq n$ ($i \neq j$), Bobo defines their *friend index* $\text{Friend}(i, j)$ and *enemy index* $\text{Enemy}(i, j)$ respectively as follows:

$$\text{Friend}(i, j) \triangleq \max(|x_i - x_j|, |y_i - y_j|), \quad \text{Enemy}(i, j) \triangleq |x_i - x_j| + |y_i - y_j|.$$

For any $1 \leq i, j \leq n$ ($i \neq j$), Bobo calls the j -th person is a *best friend* of the i -th person if

$$\text{for all } 1 \leq k \leq n \ (k \neq i), \quad \text{Friend}(i, k) \geq \text{Friend}(i, j).$$

Also, for any $1 \leq i, j \leq n$ ($i \neq j$), Bobo calls the j -th person is a *worst enemy* of the i -th person if

$$\text{for all } 1 \leq k \leq n \ (k \neq i), \quad \text{Enemy}(i, k) \leq \text{Enemy}(i, j).$$

Now Bobo wants to find out, for each $1 \leq t \leq n$, how many ordered pairs (i, j) satisfy $1 \leq i, j \leq t$, $i \neq j$, and the j -th person is both a best friend and a worst enemy of the i -th person if we only consider the first t people.

Please be aware of the unusual memory limit.

Input

The first line contains one integer, n ($2 \leq n \leq 4 \times 10^5$).

The next n lines each contain two space-separated integers, where the i -th line contains x_i and y_i ($1 \leq x_i, y_i \leq 10^7$). It is guaranteed that for $i \neq j$, either $x_i \neq x_j$ or $y_i \neq y_j$.

Output

Output n lines, each line containing a single integer, denoting the number of pairs that meet the requirements.

Examples

standard input	standard output
2 1 5 1 10	0 2
4 2 5 5 3 5 7 8 5	0 2 4 4
9 3 4 3 6 4 3 4 7 5 5 6 3 6 7 7 4 7 6	0 2 1 0 4 5 6 7 8
13 3 5 4 4 4 5 4 6 5 3 5 4 5 5 5 6 5 7 6 4 6 5 6 6 7 5	0 2 4 7 2 2 5 2 2 3 3 4 4

Note

In the first example, when considering only the first person, there are no ordered pairs that meet the requirements. When considering the first two people, there are two ordered pairs that meet the requirements: (1, 2) and (2, 1).

In the second example, when considering only the first person, there are no ordered pairs that meet the requirements. When considering the first two people, there are two ordered pairs that meet the requirements: (1, 2) and (2, 1). When considering the first three people, there are four ordered pairs that meet the requirements: (1, 2), (1, 3), (2, 1), and (3, 1). When considering the first four people, there are four ordered pairs that meet the requirements: (2, 1), (2, 4), (3, 1), and (3, 4).

Problem E. 覆盖 / Pokrycia

Time Limit 7000 ms

Mem Limit 524288 kB

Background

译自 [Potyczki Algorytmiczne 2016](#) R5 Pokrycia [A] (POK)。

Description

简单无向图 $G = (V, E)$ 的**点覆盖**是一个点集 $S \subseteq V$ ，使得 $\forall (u, v) \in E$ ，都有 $u \in S$ 或 $v \in S$ 。点覆盖 S 的大小定义为 $|S|$ 。

给定点集 V 和整数 k ，求出有多少张以 V 为点集的简单无向图 G 的最小点覆盖大小为 k 。

两张图 $G_1 = (V, E_1)$ 和 $G_2 = (V, E_2)$ 不同，当且仅当存在 $u, v \in V$ ，使得 (u, v) 只属于 E_1 或 E_2 。

给定正整数 n ，点集 $V = \{1, 2, \dots, n\}$ 。

由于答案可能很大，所以只需要输出答案模 **2** 后的余数。

Input

本题单个测试点内有多组测试数据。

第一行一个正整数 T ，表示测试数据组数。

接下来描述 T 组测试数据：

每组测试数据只有一行两个整数 n, k ，表示一次询问。 $V = \{1, 2, \dots, n\}$ 。

Output

输出 T 行，每行一个非负整数，表示答案模 **2** 后的余数。

Sample 1

Input	Output
4 3 1 5 4 5 3 57 32	0 1 1 1

Hint

样例解释

- 第一组测试数据中， $n = 3, k = 1$ 。符合条件的图要么只有一条边，要么有两条边，且这两条边共用一个顶点。不难验证，原始答案为 6。
- 第二组测试数据中， $n = 5, k = 4$ 。不难验证符合条件的图只有完全图。

数据范围

- $1 \leq T \leq 2^{14}$;
- $1 \leq n < 2^{14}$;
- $0 \leq k < n$ 。

Problem F. 采矿

Time Limit 1000 ms

Mem Limit 262144 kB

Background

在精心地规划完工人的移动路线，执行完所有计划之后，你终于有钱了。你承包下了一个更大的矿坑，并购买了更先进的设备。

但是开始运行了你才发现，一部分运输矿物的通道居然装反了！还好它们本来就是可以反向的，并且中控的系统可以让你轻易地操作。

然而，现在最大的问题是，你刚刚接手这个矿坑，你甚至连它长什么样都不知道，也就更不知道每个开关是对应哪一条运输通道的。

时间就是金钱，你想要尽快摸清整个矿坑的结构以及所有开关与通道的对应关系。

Description

这是一道交互题。

已知你的矿坑有 n 个节点，编号为 $1 \sim n$ 。它们通过 $n - 1$ 条运输通道连成一个树形结构。

运输通道都是单向的。对于一条从节点 u 到节点 v 的运输通道，可以将所有由节点 u 产出的矿或运送到节点 u 的矿以较快的速度运送到节点 v 。如果一个节点有多条以其为起点的运输通道，那么会把这些矿平均分配给这些运输通道。

中控的系统包含 $n - 1$ 个开关和一个监视器。开关的编号为 $1 \sim n - 1$ ，每个开关可以拨到 0 或 1 的位置。 $n - 1$ 个开关和 $n - 1$ 条运输通道一一对应，但你并不知道它们的对应关系。你只知道，假设编号为 i 的开关对应的运输通道在被装上去时是从 u_i 到 v_i 的，那么当开关拨到 0 的时候，它的运输方向和它被装上去时相同；当开关拨到 1 的时候，它的运输方向会变成从 v_i 到 u_i 。你的监视器可以监控到达每个节点的矿分别来自多少个不同的节点，也就是说，有多少个节点（包括其本身）能够通过运输通道把矿运输到这个节点。

当你调整完开关的位置后，需要等一段时间，监视器的结果才会趋于稳定，这时你的读数才是有意义的。所以为了避免浪费太多时间，你希望在 50 次读数之内确定你想知道的所有信息。

Input

开始时你需要输入一个正整数 n 。保证 $2 \leq n \leq 10000$ 。

之后的输入会基于你的输出生成读数。你的读数结果是一行 n 个正整数，其中第 i 个表示到达节点 i 的矿来自多少个不同的节点。

每个测试点中，矿坑的连接方式和所有运输通道刚装上去时的方向都是固定的，也就是说，这些不会因为你的输出而动态修改为另外一种符合之前所有回答的方案。

Output

当你需要调整开关并等待读数时，输出一行 `? s`，其中 `s` 为一个长为 $n - 1$ 的 `01` 串，其中第 i 位表示编号为 i 的开关拨到的位置。然后交互库会在你的标准输入中给出监视器趋于稳定之后的读数结果。你最多只能读数 50 次。

当你已经知道了所有通道的信息时，输出一行 `! u1 v1 ... un-1 vn-1`，其中 u_i, v_i 表示编号为 i 的开关对应的运输通道被装上去时的方向是从节点 u_i 到节点 v_i 的。

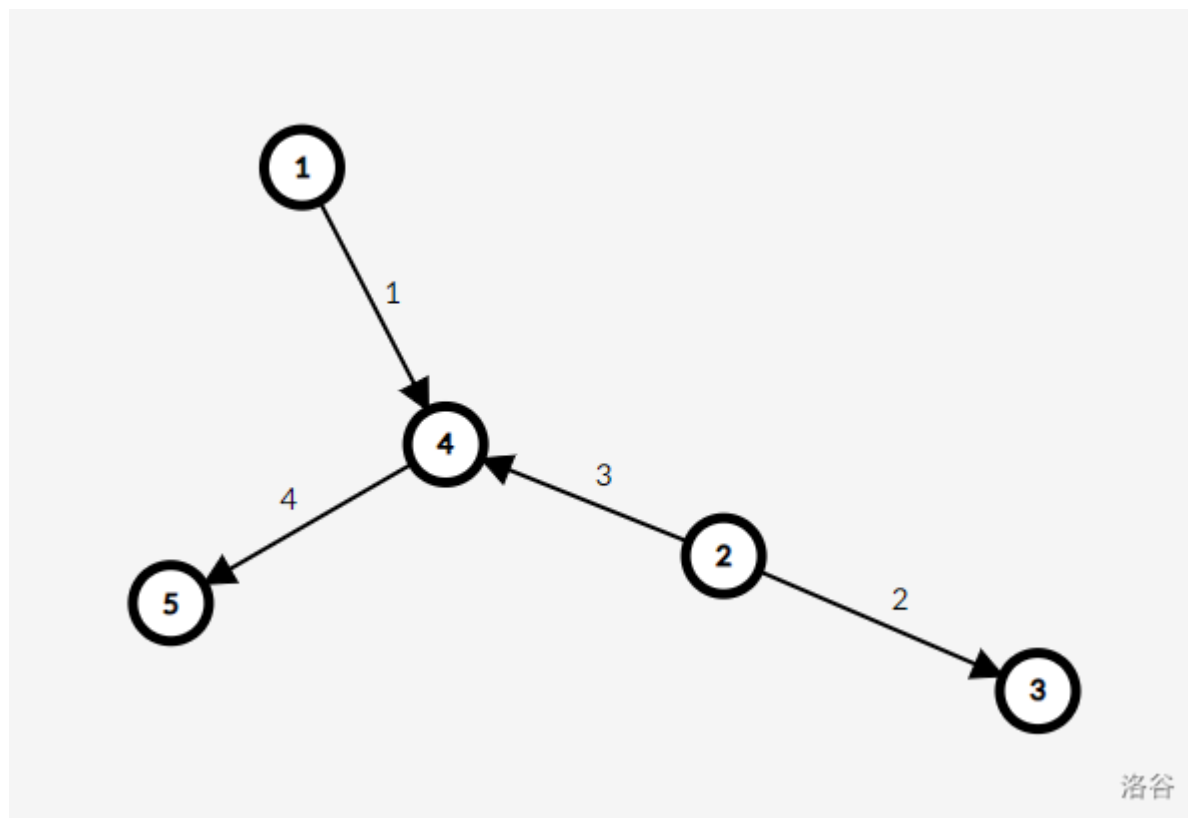
在输出一行之后，你需要刷新输出缓冲区，否则评测结果可能会变成 `TLE`。刷新输出缓冲区的方式为：

- C : `fflush(stdout);`
- C++ : `fflush(stdout);` 或 `std::cout.flush();` 或使用 `std::endl` 换行
- Java : `System.out.flush();`
- Python : `sys.stdout.flush()`

Sample 1

Input	Output
5	? 0110
1 4 1 2 3	? 0000
1 1 2 3 4	! 1 4 2 3 2 4 4 5

Hint



通道的初始方向如上图所示。通道上的数字代表和通道对应的开关的编号。

样例只是用来说明输入输出格式和读数结果，并不意味着这次读数能够推出答案。

交互库的运行时间和内存不计入时间和内存限制。

若超出读数次数限制、最后的回答错误或输出格式错误，评测结果均为 **WA**。

来源与致谢

来自 THUPC2024（2024年清华大学学生程序设计竞赛暨高校邀请赛）决赛。感谢 THUSAA 的提供的题目。

数据、题面、标程、题解等请参阅 THUPC 官方仓库 <https://thusaac.com/public>

Dumb Problem

Input file: *standard input*
Output file: *standard output*
Time limit: 1 second
Memory limit: 1024 mebibytes

You are given an integer sequence of length n : $a_1, a_2, \dots, a_n$. You can perform the following operation exactly once:

- Choose a permutation $p_1, p_2, \dots, p_n$ of $1, 2, \dots, n$.
- Then, for i from 1 to n in the natural order, execute:

$$a_{p_i} \leftarrow a_{p_i} + a_{p_i-1} + a_{p_i+1},$$

where $a_0 = 0$ and $a_{n+1} = 0$.

Your goal is to maximize the sum of all elements in the sequence after the operation is completed.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$), the number of test cases. For each test case:

The first line contains an integer n ($1 \leq n \leq 2 \cdot 10^5$).

The second line contains n integers $a_1, a_2, \dots, a_n$ ($-10^8 \leq a_i \leq 10^8$).

The sum of n does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single line with an integer representing the maximum possible sum of all elements in the sequence after performing the operation.

Example

<i>standard input</i>	<i>standard output</i>
4	22
3	85
1 2 3	72
4	79
-1 3 7 6	
5	
4 1 0 5 2	
6	
4 -3 7 5 -9 3	

Bipartite Graph Weighting Problem

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 1024 mebibytes

You are given a bipartite graph. The left part contains n vertices, numbered from 1 to n , and the right part also contains n vertices, numbered from 1 to n . There are m edges, each connecting one vertex on the left to one vertex on the right.

You will be asked q independent queries. In each query, you are given three integers: (a, b, k) .

For a fixed query (a, b, k) , consider the following process on the given graph. Initially, every vertex (both left and right) has weight 0. You may perform the following operation any number of times:

- Choose an arbitrary subset S of the left vertices (possibly empty), and choose a positive real number $p > 0$.
- Let $N(S)$ be the set of right vertices that are adjacent to at least one vertex in S .
- Increase the weight of every vertex in S by $a \cdot p$.
- Increase the weight of every vertex in $N(S)$ by $b \cdot p$.

Let $p_1, p_2, \dots, p_\ell$ be the values of p used in all your operations. These must satisfy

$$\sum_{i=1}^{\ell} p_i \leq 1.$$

Your goal is to choose the operations (the subsets S and the values p) so that **the total sum of the weights of all vertices** (both left and right) is at most k . Under this constraint, you should maximize the total sum of the weights of all vertices of the left part.

For each query (a, b, k) , each time starting from all vertex weights equal to 0, compute the maximum possible total weight of the left part that can be obtained.

Input

The first line contains a single integer t ($1 \leq t \leq 1000$), the number of test cases. For each test case:

The first line contains three integers: n , m , and q ($1 \leq n \leq 2000$, $0 \leq m \leq 10^4$, $1 \leq q \leq 2 \cdot 10^5$).

Each of the next m lines contains two integers, u and v ($1 \leq u, v \leq n$), denoting an edge between left vertex u and right vertex v .

Each of the next q lines contains three integers: a , b , and k ($0 \leq a, b \leq 10^6$, $0 \leq k \leq 10^9$), describing a query.

There are no multiple edges. The sum of n does not exceed 2000. The sum of m does not exceed 10^4 . The sum of q does not exceed $2 \cdot 10^5$.

Output

For each query, output a single real number: the maximum possible total sum of vertex weights of the left part that can be obtained for this query. Let your answer be a and the jury's answer be b . Your answer will be considered correct if $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$.

Example

<i>standard input</i>	<i>standard output</i>
2	1.2500000000000000
5 8 3	1.3333333333333333
1 2	2.142857142857144
1 3	2.0000000000000000
2 4	0.0000000000000000
2 5	3.0000000000000000
3 1	
3 3	
4 2	
5 4	
1 3 5	
2 1 2	
5 2 3	
2 3 3	
1 2	
1 1	
2 1	
1 0 2	
0 2 2	
3 1 4	

Many Approaches

Input file:	standard input
Output file:	standard output
Time limit:	3 seconds
Memory limit:	1024 megabytes

There is a park with N squares arranged in a row from left to right, numbered $0, 1, \dots, N - 1$ in this order.

Inside the park, there are N people, also numbered $0, 1, \dots, N - 1$. When you announce a sequence $X = (X_1, X_2, \dots, X_{|X|})$ of non-negative integers between 0 and $N - 1$, the people perform a **march** according to the following rules:

1. For each $i = 0, 1, \dots, N - 1$, person i moves to square i .
2. For each $j = 1, 2, \dots, |X|$, in this order, do the following:
 - Every person not currently on square X_j moves exactly one square toward X_j .

You are given a sequence $A = (A_0, A_1, \dots, A_{M-1})$ of length M , consisting of integers between 0 and $N - 1$.

You must answer Q online queries. For each $i = 1, 2, \dots, Q$, integers t'_i, L'_i, R'_i, P'_i are given. First, reconstruct t_i, L_i, R_i, P_i using the following procedure:

Let $\text{ans}_0 = 0$, and let ans_i denote the answer to the i -th query. Reconstruct t_i, L_i, R_i, P_i as follows:

- $t_i = ((t'_i + \text{ans}_{i-1}) \bmod 2)$
- $a = ((L'_i + \text{ans}_{i-1}) \bmod M)$
- $b = ((R'_i + \text{ans}_{i-1}) \bmod M)$
- $L_i = \min(a, b)$
- $R_i = \max(a, b)$
- $P_i = ((P'_i + \text{ans}_{i-1}) \bmod N)$

Here, for a non-negative integer a and a positive integer b , $(a \bmod b)$ denotes the remainder when a is divided by b , which is in the range 0 through $b - 1$.

For each reconstructed (t_i, L_i, R_i, P_i) , answer the following query:

- If $t_i = 0$: Let $X = (A_{L_i}, A_{L_i+1}, \dots, A_{R_i})$. Simulate the march and output the final square where person P_i ends up.
- If $t_i = 1$: Let $X = (A_{L_i}, A_{L_i+1}, \dots, A_{R_i})$. Simulate the march and output how many people end up on square P_i .

Input

The input is given in the following format:

```
N M Q
A_0 A_1 ... A_{M-1}
t'_1 L'_1 R'_1 P'_1
t'_2 L'_2 R'_2 P'_2
⋮
t'_Q L'_Q R'_Q P'_Q
```

- All input values are integers.
- $1 \leq N, M, Q \leq 2 \times 10^5$
- $0 \leq A_i \leq N - 1$ ($0 \leq i \leq M - 1$)
- $0 \leq t'_i, t_i \leq 1$ ($1 \leq i \leq Q$)
- $0 \leq L'_i, R'_i \leq M - 1$ ($1 \leq i \leq Q$)
- $0 \leq L_i \leq R_i \leq M - 1$ ($1 \leq i \leq Q$)
- $0 \leq P'_i, P_i \leq N - 1$ ($1 \leq i \leq Q$)

Output

Output Q lines. For each $i = 1, 2, \dots, Q$, output ans_i , the answer to the i -th query.

Examples

standard input	standard output
4 5 3 0 2 3 2 1 0 1 3 2 1 0 2 1 1 4 4 1	2 0 3
7 4 1 3 3 3 3 1 3 0 3	7

Note

In the first example, for $(t_i, L_i, R_i, P_i) = (0, 1, 3, 2)$ with $X = (2, 3, 2)$, person 2 moves as $2 \rightarrow 2 \rightarrow 3 \rightarrow 2$, so the answer is 2.

In the second example, for $(t_i, L_i, R_i, P_i) = (1, 2, 4, 3)$ with $X = (3, 2, 1)$, the number of people on square 3 at the end is 0.

In the third example, for $(t_i, L_i, R_i, P_i) = (1, 4, 4, 1)$ with $X = (1)$, the number of people on square 1 at the end is 3.

Grammy's Kingdom

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 512 megabytes

Grammy's kingdom is in danger. Some foreign invaders have intruded into her kingdom and they are destroying the traffic system. There are n stations indexed from 1 to n in the traffic system. Moreover, there are m ($m \leq n$) airports located in some of the stations. If you are in station i , you can transfer to station $i + 1$ if i and $i + 1$ haven't been destroyed. If you are in station i **with an airport**, you can fly to station j **if $i \leq j$ and station j has an airport** and both of the stations haven't been destroyed. We define the stability G of the system as the number of routes that still available.

Formally, $G = \sum_{1 \leq i \leq j \leq n}$ [a person in station i can transfer to station j via several stations or airports].

At each moment i ($1 \leq i \leq n$), the invaders will randomly choose an undestroyed station x and destroy it as well as the airport in it (If there exists an airport in it).

We define $E(x)$ as the expected value of x . Grammy wonders the value of $\sum_{i=1}^n E(G_i)$ modulo 998 244 353. Could you please help her? Here, G_i denotes the value of the stability G after the invaders destroying the i -th chosen station at the i -th moment.

Input

The input contains only a single case.

The first line contains two positive integers n and m ($1 \leq n \leq 2\,000\,000$, $0 \leq m \leq n$), denoting the number of stations and the number of airports.

The second line contains m distinct integers $x_1, x_2, \dots, x_m$ ($1 \leq x_i \leq n$), denoting the indexes of station that has an airport.

Output

Output one integer, the value of $\sum_{i=1}^n E(G_i)$ modulo 998 244 353.

Examples

standard input	standard output
1 0	0
3 3 1 2 3	4
6 2 2 4	16637434

Note

It can be proved that the answer can be represented as a rational number $\frac{p}{q}$ with $\gcd(p, q) = 1$. Therefore, you are asked to find the value of $pq^{-1} \bmod 998\,244\,353$. It can be shown that $q \bmod 998\,244\,353 \neq 0$ under the given constraints of the problem.

Strange Sorting 2

Input file: **standard input**
Output file: **standard output**
Time limit: 2 seconds
Memory limit: 1024 megabytes

Little Cyan Fish and Little Kevinfish are playing a game of sorting sequences. Little Kevinfish has a tree T with n vertices, where the vertices are numbered with integers from 1 to n .

For a sequence A consisting of integers from 1 to n , Little Kevinfish defines a swap operation as:

- Choose indices i, j , such that the vertices numbered A_i and A_j are directly connected by an edge in T .
- Swap the positions of A_i and A_j .

Little Kevinfish asks Little Cyan Fish the following question:

- For a given constant m , for each $\ell = 1, 2, \dots, m$, solve the following problem:
 - Consider a sequence A of length ℓ consisting of integers from 1 to n (there are n^ℓ such sequences in total), how many sequences A can be transformed into a sequence that is monotonically non-decreasing through some number of the above swap operations.

Please help Little Cyan Fish answer Little Kevinfish’s question. Since the answer may be very large, you only need to output the answer modulo $10^9 + 7$.

Input

There are multiple test cases. The first line of the input contains a single integer T ($1 \leq T$), indicating the number of test cases.

For each test case, the first line of the input contains two integers n and m ($1 \leq n \leq 200, 1 \leq m \leq 10^5$).

The next $(n - 1)$ lines each contain two integers u_i and v_i ($1 \leq u_i, v_i \leq n, u_i \neq v_i$), indicating an edge connecting vertex u_i and v_i . It is guaranteed that these $(n - 1)$ edges form a valid tree.

It is guaranteed that the sum of n over all test cases does not exceed 200, and the sum of m over all test cases does not exceed 10^5 .

Output

For each test case, output a single line containing m integers. The i -th ($1 \leq i \leq m$) integer represents the answer when $\ell = i$, modulo $10^9 + 7$.

Example

standard input	standard output
3	3 8 23 70
3 4	4 13
1 2	1 1 1 1 1 1 1 1 1 1
2 3	
4 2	
1 2	
1 3	
3 4	
1 10	